

NATIONAL UNIVERSITY OF SINGAPORE



SCHOOL OF COMPUTING
READING ASSESSMENT 1 FOR
Semester 1 AY2026/27

CS1101S Programming Methodology

4 September 2026

Time Allowed: 1 hour

INSTRUCTIONS TO CANDIDATES

1. This question paper contains 20 questions and comprises 10 printed pages, including this page.
2. You are also provided with an answer sheet on which to write your answers.
3. Use a pen or pencil to write your Student Number in the designated space on the front page of the answer sheet, and shade the corresponding circle completely in the grid for each digit or letter. **DO NOT WRITE YOUR NAME.**
4. You must submit only the answer sheet and no other documents.
5. All questions must be answered in the space provided on the answer sheet; no extra sheets will be accepted as answers.
6. For multiple-choice questions, shade in the circle of the correct answer completely.
7. Each question has one correct answer. The number of marks awarded for a correct answer is shown next to each question. There is no penalty for an incorrect answer.
8. Unless otherwise stated, all programs use the Source \$1 language.
9. The full score of this assessment is **60 marks**.
10. Answer all questions.
11. This is a **closed-book** assessment.

Section A: Iterative and Recursive Processes

1. [3 marks] Assume that x is a number and n is a non-negative integer. Consider the following two functions:

```
def wizard_power(x, n):  
    return (1  
            if n == 0  
            else x * wizard_power(x, n - 1))  
  
def robot_power(x, n):  
    def loop(base, count, product):  
        return (product  
                if count == 0  
                else loop(base, count - 1, base * product))  
    return loop(x, n, 1)
```

Consider the following claims:

Claim 1: `wizard_power(x, n)` gives rise to a recursive process.

Claim 2: `robot_power(x, n)` gives rise to an iterative process.

Which option correctly indicates whether each claim is true or false?

- (a.) Claim 1: False, Claim 2: True
 - (b.) Claim 1: False, Claim 2: False
 - (c.) Claim 1: True, Claim 2: True ✓**
 - (d.) Claim 1: True, Claim 2: False
2. [3 marks] For the same two functions `wizard_power(x, n)` and `robot_power(x, n)`, which of the following claims are true or false?

Claim 3: Both `wizard_power` and `robot_power` are recursive functions.

Claim 4: Both functions evaluate to x^n .

- (a.) Claim 3: True, Claim 4: False

(b.) Claim 3: False, Claim 4: True ✓

- (c.) Claim 3: False, Claim 4: False

- (d.) Claim 3: True, Claim 4: True

3. [3 marks] Consider the following iterative implementation of exponentiation by successive squaring:

```
def hyper_power(base, count, acc):  
    return (acc  
            if count == 0  
            else hyper_power(base * base, count // 2, acc)  
            if is_even(count)  
            else hyper_power(base, count - 1, acc * base))
```

It is initially applied as `hyper_power(x, n, 1)`. Assume that `is_even(k)` returns `True` exactly when the integer `k` is even. Which quantity remains unchanged from one application of `hyper_power` to the next?

- (a.) `acc + base * count`
 - (b.) `count`
 - (c.) `base`
 - (d.) `basen`
 - (e.) `acccount * base`
 - (f.) `acc * basecount` ✓**
 - (g.) `acc`
4. [3 marks] A spacecraft receives a positive integer security code `n`. We specify that `mirror_code(n)` should return the integer obtained by reversing the decimal digits of `n`. You may assume that the final digit of `n` is nonzero. For example, `mirror_code(1729)` should return 9271.

Consider the following function:

```
def mirror_code(n):  
    def loop(remaining, result):  
        return (result  
                if remaining == 0  
                else loop(<next_remaining>, <next_result>))  
    return loop(n, 0)
```

Which pair of expressions should replace `<next_remaining>` and `<next_result>`, respectively?

- (a.) `remaining % 10` and `10 * result + remaining // 10`
- (b.) `remaining // 10` and `result + remaining % 10`
- (c.) `remaining - 1` and `result + remaining`
- (d.) `remaining // 10` and `10 * result + remaining % 10` ✓**

5. [3 marks] An alien control panel initially displays 1. It has two buttons:

- **BOOST** adds 1 to the displayed number.
- **WARP** multiplies the displayed number by 2.

A command plan is a sequence of zero or more button presses that ends exactly at a positive integer target n , without ever exceeding n . Different sequences of button presses are considered different plans. For example, there are two command plans for reaching 3: BOOST/BOOST, WARP/BOOST.

Consider the following function:

```
def command_plans(n):  
    return (<base_case>  
        if n == 1  
        else <ME1>  
        if n % 2 == 1  
        else <ME2>)
```

Which option correctly replaces the three missing expressions so that `command_plans(n)` returns the number of command plans for reaching n ?

(a.) <base_case>: 1;
<ME1>: `command_plans(n // 2)`;
<ME2>: `command_plans(n - 1) + command_plans(n // 2)`

(b.) <base_case>: 1;
<ME1>: `command_plans(n - 1)`;
<ME2>: `command_plans(n // 2)`

(c.) <base_case>: 1;
<ME1>: `command_plans(n - 1)`;
<ME2>: `command_plans(n - 1) + command_plans(n // 2)` ✓

(d.) <base_case>: 1;
<ME1>: `1 + command_plans(n - 1)`;
<ME2>: `command_plans(n - 1) + command_plans(n // 2)`

(e.) <base_case>: 0;
<ME1>: `command_plans(n - 1)`;
<ME2>: `command_plans(n - 1) + command_plans(n // 2)`

(f.) <base_case>: 1;
<ME1>: `command_plans(n - 1)`;
<ME2>: `command_plans(n - 1) + command_plans(n - 2)`

Section B: Order of Evaluation and Correctness of Programs

6. [3 marks] What is the sequence of values printed when the following Python \$1 program is evaluated?

```
print(1)

def f(x):
    print(2)
    return x

print(3)
f(4)
```

(a.) 1 3 2 4

(b.) 1 2 3

(c.) 1 2 3 4

(d.) 2 1 3

(e.) 1 3 2 ✓

(f.) None of the other options is the correct answer.

7. [3 marks] Which one of the following statements correctly describes what happens when the following Python \$1 program is evaluated?

```
def follow(x, y):
    print(x)
    return x if x == y else follow(y, x + 1)

follow(1, 4)
```

(a.) No values are printed, and the program keeps going.

(b.) The values 1 4 2 5 3 6 4 7 ... are printed, and the program keeps going. ✓

(c.) The values 1 4 2 5 3 6 4 are printed, and the program finishes with the value 4.

(d.) The values 1 4 2 5 3 6 are printed, and the program finishes with the value 6.

(e.) The values 1 2 3 4 are printed, and the program finishes with the value 4.

(f.) None of the other options is the correct answer.

8. [3 marks] What is the sequence of values printed when the following Python \$1 program is evaluated?

```
def D(x):  
    print(x)  
    return x  
  
def combine(x, y):  
    return D(D(x + y) * D(x - y))  
  
combine(D(7), D(2))
```

- (a.) 7 9 2 5 45
 - (b.) 9 5 45 7 2
 - (c.) 7 2 9 5 45 ✓**
 - (d.) 7 2 45 9 5
 - (e.) 7 2 9 45
 - (f.) None of the other options is the correct answer.
9. [3 marks] We specify that CC, when applied to any two numbers, should return the sum of the squares of the two numbers.

```
def square(x):  
    return x * x  
  
def CC(x, y):  
    return square(square(y) + square(x))
```

Which one of the following statements is correct?

- (a.) CC does not meet the specification because an error occurs during the evaluation for every valid pair of arguments.
- (b.) CC does not meet the specification because it returns incorrect results for every valid pair of arguments.
- (c.) CC meets the specification.
- (d.) CC does not meet the specification because it returns correct results for some but not all valid pairs of arguments. ✓**
- (e.) CC does not meet the specification because it contains syntax errors.

10. [3 marks] We specify that `safe_divide`, when applied to any two numbers x and y , should return 0 if y is zero and x / y otherwise.

Consider the following three implementations.

```
def safe_divide_A(x, y):  
    return 0 if y == 0 else x / y  
  
def safe_divide_B(x, y):  
    return (0 if y == 0 else x) / (1 if y == 0 else y)  
  
def safe_divide_C(x, y):  
    return (0 if y == 0 else x) / y
```

Which of the implementations meet the specification?

- (a.) Only `safe_divide_B`
 - (b.) Only `safe_divide_A` ✓**
 - (c.) `safe_divide_A` and `safe_divide_B` only ✓**
 - (d.) `safe_divide_A` and `safe_divide_C` only
 - (e.) All three implementations
 - (f.) None of the implementations
11. [3 marks] We specify that `middle`, when applied to any three mutually distinct numbers, should return the number whose value lies between the other two values.

```
def smaller(x, y):  
    return x if x < y else y  
  
def larger(x, y):  
    return x if x > y else y  
  
def middle(x, y, z):  
    return larger(smaller(x, y), smaller(larger(x, y), z))
```

Which one of the following statements is correct?

- (a.) `middle` does not meet the specification because it returns an incorrect result for every valid set of arguments.
- (b.) `middle` does not meet the specification because it returns correct results for some but not all valid sets of arguments.
- (c.) `middle` does not meet the specification because it contains syntax errors.
- (d.) `middle` does not meet the specification because an error occurs during the evaluation for every valid set of arguments.

(e.) `middle` meets the specification. ✓

Section C: Scope and Higher-Order Functions

12. [3 marks] Consider the following function `f`:

```
def f():  
    return f()
```

What is printed by the following Python 3 program?

```
print((lambda x: lambda y: x)(1)(f))
```

- (a.) A function.
- (b.) No result: the program takes too many steps.
- (c.) No result: `NameNotFoundError`.
- (d.) 1 ✓**
- (e.) No result: `SelfApplication`.

13. [3 marks] Consider the following function `f`:

```
def f():  
    return f()
```

What is printed by the following Python 3 program?

```
print((lambda x: lambda y: x)(1)(f()))
```

- (a.) No result: the program takes too many steps. ✓**
- (b.) No result: `NameNotFoundError`.
- (c.) A function.
- (d.) 1
- (e.) No result: `SelfApplication`.

14. [3 marks] Consider the following function `f`:

```
def f():  
    return f()
```

What is printed by the following Python 3 program?

```
print((lambda x: lambda y: x)(1)(lambda: f()))
```

- (a.) No result: the program takes too many steps.
- (b.) A function.
- (c.) No result: `NameNotFoundError`.
- (d.) 1 ✓**
- (e.) No result: `SelfApplication`.

15. [3 marks] Consider the following function `repeat_apply`:

```
def repeat_apply(t, n, x):  
    return x if n == 0 else repeat_apply(t, n - 1, t(x))
```

What is printed by the following Python 3 program?

```
print(repeat_apply(lambda n: n + 1, 3, 4))
```

- (a.) A `NameNotFoundError`
 - (b.) The number 3
 - (c.) The number 4
 - (d.) The number 7 ✓**
 - (e.) A function
16. [3 marks] Consider the following function `repeat`:

```
def repeat(t, n):  
    return lambda x: x if n == 0 else repeat(t, n - 1)(t(x))
```

What is printed by the following Python 3 program?

```
print(repeat(lambda n: n + 1, 3)(4))
```

- (a.) The number 3
 - (b.) The number 4
 - (c.) A `NameNotFoundError`
 - (d.) A function
 - (e.) The number 7 ✓**
17. [3 marks] Consider the following function `make_repeat`:

```
def make_repeat(t):  
    return lambda n: lambda x: (  
        x if n == 0 else make_repeat(t)(n - 1)(t(x))  
    )
```

What is printed by the following Python 3 program?

```
print(make_repeat(lambda n: n + 1)(3))
```

- (a.) The number 7
- (b.) A `NameNotFoundError`
- (c.) The number 3
- (d.) The number 4
- (e.) A function ✓**

18. [3 marks] What is printed by the following Python \$1 program?

```
print((lambda x: x(x)(x))(lambda x: x))
```

(a.) No result: the program takes too many steps.

(b.) A function. ✓

(c.) No result: the program has too many deferred operations.

(d.) No result: NameNotFoundError.

(e.) A number.

19. [3 marks] What is printed by the following Python \$1 program?

```
print((lambda x: x(x))(lambda x: x(x)))
```

(a.) A function.

(b.) No result: NameNotFoundError.

(c.) No result: the program has too many deferred operations.

(d.) A number.

(e.) No result: the program takes too many steps. ✓

20. [3 marks] What is printed by the following Python \$1 program?

```
print((lambda x: x(x))(lambda x: x(x)(x)))
```

(a.) No result: NameNotFoundError.

(b.) No result: the program has too many deferred operations. ✓

(c.) A function.

(d.) No result: SelfApplication.

(e.) A number.

END OF PAPER